Day 60 - Performance Tuning

Objective. Improve scoring speed, support multiple resumes per service call, assess memory and cache use, and define a path to scale the system. This report records the local benchmark and the limits of the current repository.

Optimizations completed

Vectorized skill comparison. Skill embeddings are now collected once per scoring call and pairwise cosine scores are calculated as a matrix. The shared SentenceTransformer instance and bounded 256-entry embedding LRU cache remain in place.

Batch service method. `CandidateMatchingService.generate_scores(resumes, job_description)` accepts an iterable, scores resumes sequentially, and returns results in input order. It reuses the existing model and cache; it does not batch model encoding across resumes.

Warm scoring benchmark

Eight labeled resumes were scored using the same local service benchmark. Baseline warm scoring was 0.7371 seconds total, averaging 0.0921 seconds per resume. After vectorization, it was 0.2943 seconds total, averaging 0.0368 seconds per resume - about 60% less time, or 2.5 times the throughput. Model loading and warm-up are excluded from these totals.

Score comparison. Most displayed scores stayed the same. CV ASIF changed from 52.36% to 51.97%, cv data from 69.59% to 69.32%, and cv-2 from 69.88% to 69.78%. These small shifts appeared with the vectorized cosine calculation; review score tolerance before production use.

API latency limitation. FastAPI is a dependency, but the repository has no HTTP route or deployment configuration. The 0.0368-second measurement is service scoring time, not end-to-end HTTP response latency.

Memory footprint

The existing tracemalloc benchmark reported 250.79 KB current and 332.40 KB peak after eight resumes, but tracing starts after model initialization and does not show the full process footprint. A Windows process measurement showed a 398.66 MB working set after imports, 419.78 MB after model load, and 482.05 MB after scoring eight resumes.

Load and scaling

Cache capacity

The cache held 51 embeddings out of its 256-entry limit. A 384-dimensional float32 embedding is about 1.5 KB, so 51 entries use roughly 0.08 MB and the full cache roughly 0.38 MB. The cap was left unchanged because reducing it would save little memory and could increase misses for varied jobs and resumes.

In-process load simulation

The model was loaded before the simulation. Warm load repeated 24 cached scoring calls: one worker reached 586 requests/second with 2.1 ms p95 latency; two workers reached 466 requests/second with 8.9 ms p95; four reached 428 requests/second with 13.0 ms p95.

Cold-embedding-cache load scored eight fixtures: one worker reached 12.60 requests/second with 259 ms p95 latency; two reached 13.18 requests/second with 385 ms p95; four reached 6.85 requests/second with 1.09 seconds p95. There were no errors. Cache misses rose from 51 to 54 and 62 as concurrency increased, consistent with duplicate work on simultaneous misses.

These are small service-only diagnostics, not API capacity figures. The warm case is especially cache-heavy. More threads did not improve tail latency, and four-worker cold load degraded throughput.

Horizontal scaling strategy

When an HTTP API is added, run a stateless FastAPI service behind a load balancer and scale replicas rather than increasing threads in one model process. Load the pinned model at startup and mark a replica ready only after initialization. Keep the local LRU cache initially; each replica has its own model and cache, so sticky sessions are unnecessary. Send large batches through a bounded queue, and autoscale using CPU, queue depth, and p95 latency. Validate memory limits and HTTP latency under a real load test before setting production capacity.